

NexusTiq24 - Problem Statements

1. The problem statements

1. Healthcare - Patient Intake Triage Assistant (TRACK_ID=PS01)

Patients arriving at intake describe their situation in incomplete, everyday language - and the cost of routing them wrongly is measured in outcomes, not minutes.

Build a triage assistant that takes a patient's description in plain language, asks relevant follow-up questions when information is missing, and checks the case against a small set of triage rules covering common walk-in complaints - fever, injury, chest pain, breathing difficulty, abdominal pain. It produces a triage note: the recommended urgency level and department, the specific rule or reasoning behind the recommendation, what the patient reported versus what the follow-ups established, and what remains unknown. The system must not diagnose, must cite the rule behind every recommendation, and must escalate uncertain or high-risk cases to a human rather than guessing.

2. Insurance - Claims Evidence Review Assistant (TRACK_ID=PS02)

A claims investigator's day is spent assembling evidence before any decision can be made - forms, invoices, receipts and policy wordings that rarely agree with each other.

Build an evidence review assistant for motor insurance claims - two-wheelers and cars that are damaged in an accident or stolen. The system takes a claim: the claim form, the repair estimate or FIR, and the customer's incident description, and reviews it against a small motor policy - what is covered, what is excluded, the insured value, claim windows and required documents. It produces a claim review: whether the submitted documents are complete and consistent, which policy clauses apply and whether they support or block the claim, every finding cited to the document and clause it came from, and a recommendation - approve, reject, or request specific missing information. Contradictions between documents must be surfaced, not smoothed over, and uncertain cases must be escalated to the investigator rather than decided.

3. Retail - Sales and Inventory Copilot (TRACK_ID=PS03)

A store manager's data is rich; their decisions are rushed. What needs attention today is buried across sales reports and stock sheets.

Build a copilot for a store manager running a small retail operation - a few stores, a catalogue of products, daily sales and stock data that your system loads and works over. The manager asks questions in plain language - what is running out, what is overstocked, how did a product do this month - and the copilot answers with the actual numbers behind every answer, never a claim without the figures. It flags what needs attention today: likely stock-outs before they happen, stock that is not moving, and sales spikes or drops worth a look - and recommends an action for each, showing the data and assumptions behind the recommendation. When the data cannot answer a question, it says so rather than guessing.

4. Customer Operations - Customer Support Resolution Assistant (TRACK_ID=PS04)

Routine requests clog the support queue while complex cases wait behind them - and every handover loses context.

Build a resolution assistant for a broadband and mobile provider's support desk. A customer request comes in - a billing complaint, a connection problem, a plan question - and the assistant works it using three things: the conversation so far, the customer's account record (plan, billing status, recent tickets), and a set of support articles your system works over. For routine requests it drafts a resolution grounded in the matching article, cited, ready for an agent to approve and send. When information is missing it asks for exactly what it needs. And when the case is complex, uncertain, or not covered by any article, it transfers to a human - handing over a concise summary of the issue, what has been established, and what has already been tried, so the customer never repeats themselves.

5. Legal Operations - Lease Agreement Review Assistant (TRACK_ID=PS05)

Nobody reads the agreement until something goes wrong - and by then, the clause was always there.

Build a review assistant for the legal desk of a property management company that signs dozens of lease agreements a month. The input is a lease agreement as a document, reviewed against the company's

standard positions your system works over: acceptable ranges for deposit and notice period, required clauses like maintenance responsibility and return-of-deposit timelines, and terms the company never accepts. The system reads the agreement clause by clause and produces a review report: where the agreement matches the standard, where it deviates - quoting the exact clause and stating the deviation plainly - which required protections are missing entirely, and a plain-language summary of the three or four terms a signer most needs to understand. Every finding must quote the clause it comes from; silence in the agreement is a finding too, not a gap to skip. An agreement that matches the standards should come back clean. The report goes to a human reviewer - the system flags and explains; it does not approve.

6. Banking - Transaction Risk Investigation Assistant (TRACK_ID=PS06)

A bank reviews thousands of customer histories, and most are perfectly routine - the skill is telling the difference, and explaining it.

Build an investigation assistant for a bank's fraud desk. The input is a single customer's transaction history - date, description, payee, amount, channel - covering several months of activity, which may be entirely routine or may contain activity worth a closer look. The system reviews it against a small set of risk rules - unusually large transfers, bursts of payments to a newly added payee, odd-hours activity, transactions that break the customer's own established pattern - and produces an investigation report whose first finding is whether anything needs attention at all. When something does, the report lays out the specific transactions involved and how they connect, which rule was triggered, how the activity differs from this customer's normal behaviour, and what an investigator should look at first. When nothing does, it says so plainly - a system that finds suspicion everywhere is as useless as one that finds none. Every transaction cited must be traceable to the input history, and the system must never state that fraud has occurred - it flags, explains, and hands the judgement to the investigator.

7. Telecom - Network Incident Triage Assistant (TRACK_ID=PS07)

When something breaks in a network, it never raises one alert - it raises fifty, and someone has to work out that they are one problem.

Build a triage assistant for a network operations desk. The input is a stream of alerts and log events of familiar kinds - a link going down, a device unreachable, high latency, repeated authentication failures - where one underlying failure typically produces many overlapping signals. The system groups duplicates and related alerts into incidents, prioritises them by likely impact, and for each incident consults a small set of troubleshooting runbooks your system works over to recommend an initial response - citing the runbook it drew from. Alerts that do not belong to any incident are left as noise, not forced into one. When an incident is not covered by any runbook, the system escalates it with the context assembled so far - what happened, what was grouped together, what was already suggested - so the next team starts from evidence, not from scratch.

8. Supply Chain - Disruption Response Assistant (TRACK_ID=PS08)

When a shipment slips or a supplier fails, the delay is rarely in reacting - it is in working out what the disruption actually affects.

Build a response assistant for a distributor that buys from suppliers, holds stock in a warehouse, and fulfils customer orders - working over your system's own data for suppliers, stock, shipments and orders. The input is a disruption notice as it actually arrives - a supplier's email about a production halt, a carrier's delay notification, a warehouse incident report - unstructured text that names things loosely. The system must first work out what the notice means and which suppliers, shipments or stock it maps to in the data, then trace what it actually affects - which stock runs short, which orders slip, which customers are hit and by when - and produce an impact assessment and action plan: the affected orders ranked by urgency, the options for each with trade-offs stated plainly - expedite at higher cost, part-ship, reallocate stock, or inform the customer of a new date - and a recommended course, with every impact claim traceable to the underlying data. A notice that sounds alarming but maps to nothing pending should come back as exactly that - no impact. The plan is prepared for a human operator - the system recommends; it does not act.

2. Evaluation criteria

- **A working application** - python app.py starts and the app comes up on port 8000 within the timeout.
- **Real commit history** - Commit history shows real progression across the 24 hours - not one giant dump at the end
- **Sound engineering** - clean, modular code with a clear separation between LLM reasoning and deterministic logic, sensible data handling, and graceful behaviour when a model call or input fails.
- **A working solution to the stated problem** - the application runs, does what the problem statement asks end to end, and handles the incomplete, ambiguous and edge cases the statement calls out rather than only the happy path.
- **A well-grounded GenAI implementation** - the AI capability grounds its answers in the system's own data and documents, with evidence or citations for what it recommends, structured and usable outputs, and the discipline to escalate or refuse rather than invent when the information is not there.
- **Problem understanding and judgement** - the solution reflects a real reading of the problem, and the design choices - what to automate, what to leave to a human, what to leave out - show considered trade-offs.

3. What you build and submit

The backend must be Python. Frontend, libraries, frameworks and GenAI tooling are your choice - we evaluate what the system does and how it is engineered.

What you must ship: a working application a judge can run and interact with, in a GitHub repository, with a 2-3 minute demo video showing the system handling a normal case and a difficult one.

Submission rules

One command to run. From the repository root: pip install -r requirements.txt, then python app.py starts your entire application - backend and frontend together - serving at http://localhost:8000. If you build a separate frontend (React, Vue, etc.), commit the built files and serve them from your Python app; a judge will not run a second command. No second terminal, no build step, no manual setup, and nothing that waits for a keypress.

Gemini is the only external API. Use the Gemini API for both your LLM calls and your embeddings (gemini-embedding-001). Read the key from the GEMINI_API_KEY environment variable - we supply our own key when we run your app - and never commit a key to the repository. Nothing else goes over the network: no other providers, no hosted vector databases, no 3rd party RAG or memory services. Local libraries like FAISS, Chroma in local mode, numpy and sqlite are fine, so build your own retrieval pipeline.

Your repository, in a fixed shape. app.py, requirements.txt and README.md at the root; everything else is your choice. **The first line of your README is your track, exactly like this: TRACK_ID=PS6** - nothing else on that line. Commit your generated data and your built frontend files; leave out virtual environments, model weights and keys.

A README a judge can follow. After the track ID line: what the project does, how to run it, what data and documents you generated, and the demo video link.

Your own data and documents. No datasets or documents are provided. Each problem statement names the material your system works over - the rules, policies, records or articles - and you create it yourself. Using an LLM to generate it is expected; a small, well-made set is enough, and your AI capability is judged on how well it grounds in it.

It has to run inside our limits. We run Python 3.11 on a clean machine. pip install gets 10 minutes, your app gets 90 seconds to start answering on port 8000, and any single request gets 60 seconds. Do your indexing or embedding at startup within that window - if it is slow, precompute the index and commit it. Test a fresh clone before you submit.

On Devfolio you submit two things: your GitHub repository link and a 5 minute demo video of the working application.

Projects that do not start this way cannot receive functional credit - so test a fresh clone of your own repository before submitting.

Example submission

A typical repository root:

```
your-project/  
  app.py          <- python app.py starts everything, on port 8000  
  requirements.txt <- all Python dependencies  
  README.md       <- what it does, how to run, env var names, demo video link  
  src/            <- your backend modules  
  frontend/dist/  <- built frontend files, if you have a separate frontend  
  data/           <- the sample data and documents your system works over
```

Folder names inside the repo are up to you - what is fixed is app.py and requirements.txt at the root, and the port.